

Aula 6 — Performance, Observabilidade & IA on-Device

"IA on-device é hype?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 09/11/2026 · Última aula

Recap das 5 aulas

1. Fundamentos & ADR
2. RN New Architecture
3. Cross-platform + bridging
4. PWA, SW, offline-first
5. KMP, BFF, GraphQL

Hoje: fechar o ciclo + apresentações dos projetos finais.

Agenda da aula (3h30)

1. **Bloco 1 (55min)** — Performance + observabilidade
2. **Bloco 2 (55min)** — IA on-device (demo Llama 3.2 ao vivo)
3. **Intervalo (30min)**
4. **Bloco 3 (55min)** — **Apresentações dos projetos finais** (grupos, 10min + 5min Q&A cada)
5. **Esquenta (15min)** — Encerramento + feedback

Objetivos de aprendizagem

- **Diagnosticar** performance em apps RN production (cold start, FPS, bundle)
- **Instrumentar** observabilidade ponta a ponta (Sentry + W3C Trace Context)
- **Executar** modelos de IA on-device em mobile (TFLite, MLC-LLM)
- **Avaliar** trade-offs on-device vs cloud com base em latência, energia e privacidade
- **Apresentar** decisões arquiteturais defensavelmente em 5 minutos

Performance RN — métricas que importam

Métrica	Target	Ferramenta
Cold start	< 2s	Firebase Performance, Sentry
FPS sustentado	60 (ou 120)	Flipper Performance Monitor
TTI (Time to Interactive)	< 3s	React DevTools Profiler
Bundle size	< 5MB JS	Source Map Explorer
Memory	< 200MB heap	Xcode Instruments / Android Profiler
Crash-free users	> 99.5%	Sentry / Crashlytics

Sem métrica, é palpite.

Performance — bundle size

```
# Gerar bundle de produção
npx react-native bundle \
  --platform ios \
  --dev false \
  --entry-file index.js \
  --bundle-output ios-bundle.js \
  --sourcemap-output ios.map

# Analisar
npx source-map-explorer ios-bundle.js ios.map
```

Otimizações:

- **Hermes** ativado (bytecode pré-compilado)
- **Tree-shaking** via metro config
- **Inline requires** (carregamento lazy)
- **RAM bundles** para apps grandes

Performance — FlatList tuning

```
<FlatList
  data={items}
  keyExtractor={(item) => item.id}
  renderItem={renderItem}
  // Otimizações:
  getItemLayout={(_, i) => ({ length: 80, offset: 80 * i, index: i })}
  initialNumToRender={10}
  maxToRenderPerBatch={5}
  windowSize={11}
  removeClippedSubviews={true}
  // Reuso de memória:
  // FlashList (Shopify) é alternativa superior para listas grandes
/>
```

Para listas com 1000+ itens, considere **FlashList** da Shopify.

Performance — `React.memo`, `useMemo`, `useCallback`

Use:

-  Componente puro com props complexas que rerenderiza desnecessariamente
-  Cálculo caro em `useMemo` (sort de 10k itens, parse JSON)
-  Callback passado pra componente memoizado

Não use:

-  Em todo lugar "por garantia" — overhead supera ganho
-  Componente leve com props primitivas
-  Antes de medir o problema

Prematura otimização é raiz de muito código ilegível.

Observabilidade — Sentry RN

```
import * as Sentry from '@sentry/react-native';

Sentry.init({
  dsn: 'https://abc123@sentry.io/12345',
  enableTracing: true,
  tracesSampleRate: 0.2, // 20% das transações
  environment: 'production',
  release: 'myapp@1.0.0',
});

// Capturar erro manualmente
try {
  await riskyOperation();
} catch (e) {
  Sentry.captureException(e, {
    tags: { feature: 'checkout' },
    extra: { userId: user.id },
  });
}

// Performance
const transaction = Sentry.startTransaction({ name: 'CheckoutFlow' });
// ... operações ...
transaction.finish();
```

Distributed tracing — W3C Trace Context

App mobile → API Gateway → Microserviço A → Microserviço B.

Erro em B reportado. Como saber qual chamada do mobile causou?

W3C Trace Context define `traceparent` header propagado por toda cadeia.

```
traceparent: 00-{trace_id}-{span_id}-{flags}
```

Ferramentas (Sentry, Datadog, OTel) montam árvore completa:

```
[mobile cold start] → [HTTPS GET /checkout] → [API Gateway]
                                                    → [Service A]
                                                    → [Service B] ← ERRO
```

OpenTelemetry mobile

```
import { trace } from '@opentelemetry/api';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';

const tracer = trace.getTracer('my-app');

async function loadFeed() {
  const span = tracer.startSpan('loadFeed');
  try {
    const data = await api.getFeed();
    span.setAttribute('feed.itemCount', data.length);
    return data;
  } catch (e) {
    span.recordException(e);
    throw e;
  } finally {
    span.end();
  }
}
```

IA on-device — por que importa em 2026

Vantagens:

-  **Privacidade** — dado não sai do device
-  **Latência** — sem round-trip de rede
-  **Offline** — funciona sem conectividade
-  **Custo zero por inferência** (após download)

Trade-offs:

-  Modelo menor (1B–8B vs 100B+ cloud)
-  Bateria + storage
-  Setup técnico mais complexo

Quando faz sentido: features sensíveis a privacidade (e-mail, healthcare, mensagens) + tarefas que não exigem SOTA.

TensorFlow Lite (TFLite)

Para modelos de visão, NLP simples, classificação on-device.

```
import { TensorflowModel } from 'react-native-fast-tflite';  
  
const model = await TensorflowModel.loadFromFile('mobilenet.tflite');  
  
const inputTensor = preprocessImage(image);  
const output = model.runSync([inputTensor]);  
const predictions = postprocess(output[0]);
```

MobileNet, EfficientNet, BERT-tiny, distilled models — todos rodam suavemente em devices modernos.

Core ML (iOS)

Apple ecosystem. Otimização específica para Neural Engine.

```
import CoreML

let model = try MyImageClassifier(configuration: .init())
let prediction = try model.prediction(image: pixelBuffer)
```

Em RN via TurboModule:

```
const result = await NativeMLClassifier.classify(imageUri);
```

Modelos da Apple (Foundation Models, iOS 18+) acessíveis via framework oficial.

MLC-LLM — LLMs em mobile

Universal LLM Deployment (mlc.ai). Roda Llama, Phi, Gemma quantizados em iOS + Android.

```
# Instalar
git clone https://github.com/mlc-ai/mlc-llm
cd mlc-llm

# Compilar Llama 3.2 1B q4f16 para iOS
mlc_llm compile dist/Llama-3.2-1B-q4f16_1-MLC \
  --device iphone \
  -o dist/Llama-3.2-1B-q4f16_1-iphone.so
```

Em iPhone 15 Pro: Llama 3.2 1B q4 → ~30 tokens/s, latência aceitável pra UX conversacional.

Quantização — int4, int8, fp16

Precisão	Tamanho (Llama 7B)	Qualidade	Uso
fp32	~28 GB	100% baseline	Treino
fp16	~14 GB	~99%	Inferência cloud
int8	~7 GB	~96%	Mobile high-end
int4	~3.5 GB	~93%	Mobile mid/low
int4 GPTQ	~3.5 GB	~96%	Mobile (recomendado)

GPTQ (Frantar 2022) preserva qualidade melhor que int4 ingênuo.

llama.cpp via JNI

Para casos onde MLC-LLM não cobre.

- llama.cpp em C++
- JNI bridge para Android
- Swift bridge para iOS
- Modelos GGUF (formato consagrado)

| Mais trabalho de bridging que MLC-LLM, mas máxima flexibilidade.

Apple Foundation Models (iOS 18+)

Apple expõe modelos pré-instalados via framework oficial.

```
import FoundationModels

let model = await FoundationModelSession()
let result = try await model.respond(to: "Sumarize: \((longText)")
```

Vantagens:

- Modelo já no device (não inflar bundle)
- Neural Engine otimizado
- Privacidade by design

Trade-off:

- Restrito a tasks suportadas
- Lock-in iOS

Gemini Nano (Android)

Análogo da Apple no Android. Modelo on-device do Google.

- Disponível em Pixel 8 Pro+ e devices selecionados
- API Java via AICore
- Tasks: summarize, rewrite, smart reply

Em 2026, foundation models on-device são commodity em flagships.

Demo ao vivo — Llama 3.2 1B no iPhone

1. Abrir app demo no iPhone (modo airplane)
2. Carregar modelo Llama 3.2 1B q4 (já bundled)
3. Prompt: *"Sumarize este e-mail em 2 frases:"* + texto
4. Aluno cronometra latência
5. Resposta gerada **sem internet**, sem nenhum dado saindo do device

| Sua sumarização de e-mail nunca mais precisa sair do device.

Federated learning mobile

Treinar modelo distribuído sem centralizar dados.

- Cada device treina localmente com seus dados
- Manda **gradientes** (não dados) pra servidor
- Servidor agrega e devolve modelo atualizado

Casos: keyboard prediction (Apple, Google), reconhecimento de voz personalizado.

| Em 2026 ainda emergente em mobile genérico, mas conceito crítico.

Tendências — onde mobile vai em 2026+

- **Edge AI** — modelos cada vez maiores rodando no device
- **WebGPU em RN-Web** — acelera ML em PWA
- **Server Components RN** — RSC chegando em mobile
- **Privacy-first by default** — regulação (LGPD, EU AI Act) força arquiteturas locais
- **App agents** — agentes LLM operando o app pelo usuário (caso de uso emergente)

Intervalo — 30min

Volta às :___

Aproveita pra:

- Esticar perna
- Conferir slides + demo do projeto final prontos pra apresentar
- Tomar café / água

Próximo bloco é **apresentações finais**. Cada aluno: 5min + 3min Q&A. Ordem por sorteio.

Apresentações — projeto final

Cada grupo: 10min demo ao vivo + 5min Q&A.

Ordem definida agora (sorteio). Rubrica (30 pts, ver `enunciado.md`):

- **CI E2E — 5 flows Maestro** (3pts/flow) — 15pts (nota mínima automática, já postada pelo bot)
- **Convenção de testID correta** — 5pts (idem, automática)
- **Contribuição individual** (commits do grupo) — 4pts (manual)
- **Apresentação + demo ao vivo + Q&A** — 4pts (manual)
- **Qualidade do código + README reproduzível** — 2pts (manual)

Nota final (auto + manual) é lançada no Canvas depois da apresentação.

Apresentações — checklist

- [] App rodando em emulador antes da apresentação (não na hora)
- [] PR do grupo com CI verde (5 flows Maestro) — confirmam antes de apresentar
- [] README reproduzível na `pratica/` do tema escolhido
- [] Todo mundo do grupo consegue falar sobre pelo menos 1 feature que implementou
- [] Capacidade de responder "por que essa escolha?" em Q&A

| Falha de domínio em Q&A vale **mais que falha técnica** na nota.

Esquenta — encerramento da disciplina

Vocês saem daqui com:

- Vocabulário arquitetural mobile preciso
- Capacidade de defender decisão de stack com critério
- Native bridging hands-on real (não só leitura)
- PWA production-grade construída
- Kotlin Multiplatform + GraphQL/BFF como padrão de integração distribuída
- IA on-device com Llama rodando em iPhone

E uma **bibliografia âncora** que vocês podem visitar nos próximos 5 anos.

Feedback (sempre bem-vindo)

Quero saber:

- O que funcionou
- O que faltou
- O que posso melhorar pra próxima oferta

Formulário: link no Canvas.

| Sua resposta vira melhoria real na oferta seguinte.

Entregáveis finais (recap)

Esta é a última aula — não há atividade assíncrona pós-aula.

Entregáveis já requeridos:

-  **Atividade 5** (KMP: Filmes Populares, 10pts) — entregue antes desta aula
-  **Projeto Final em Grupo** (30pts) — apresentado **hoje**

Notas finais publicadas no Canvas em até 7 dias.

Leituras pós-curso (para continuar aprendendo)

- React Native New Architecture continua evoluindo — **assistir RFCs no GitHub** `reactwg/react-native-new-architecture`
- Flutter Impeller em rollout completo Android — **docs.flutter.dev**
- KMP cresce em adoção (Square, JetBrains, Cash App) — **kotlinlang.org/docs/multiplatform.html**
- Foundation models on-device viram commodity — Apple WWDC keynotes, Google I/O
- Anthropic Computer Use docs — atualizadas trimestralmente

Bibliografia âncora em `BIBLIOGRAFIA.md` — revisitar nos próximos 5 anos.

Manter-se atualizado é parte do trabalho.

Obrigado!

Foi um prazer ensinar essa turma.

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith

Boa sorte com os projetos finais e nas suas decisões arquiteturais!